Crucible — Architecture

> *Interface designs for the four core engine modules. These define the contracts that all scenarios and extensions must satisfy. Implementation begins Week 1.*

Overview

```
SimSpec (data contract)
    ↓
SimRunner.setup()
    ├── builds BDIAgent instances from ActorSpec
    └── initializes TheoryBase instances from TheoryRef list
            ↓
SimRunner.run() — tick loop
    ├── apply external shocks
    ├── agents observe environment
    ├── agents update beliefs
    ├── agents decide → list[Action]
    ├── resolve actions → environment mutations
    ├── theory updates → environment mutations
    ├── record metrics
    └── trigger snapshots
```

---

1. `core/spec.py` — SimSpec

The central data contract. Output of the Scoping Agent, input to SimRunner. JSON-serializable, fully reproducible, Pydantic v2.

```python
from __future__ import annotations

import uuid
from enum import Enum
from typing import Any

from pydantic import BaseModel, Field, model_validator


# ── Belief distribution types ──────────────────────────────────────────

class BeliefDistType(str, Enum):
    BETA = "beta"          # probability beliefs (0–1), e.g. "P(actor cooperates)"
    GAUSSIAN = "gaussian"  # continuous beliefs, e.g. "estimated GDP growth"
    POINT = "point"        # known value, no uncertainty


class BeliefSpec(BaseModel):
    belief_id: str = Field(default_factory=lambda: str(uuid.uuid4()))
    name: str
```

```python
    description: str = ""
    dist_type: BeliefDistType = BeliefDistType beta
    # BETA: initial alpha and beta (both > 0)
    alpha: float = 1.0
    beta: float = 1.0
    # GAUSSIAN: initial mean and variance
    mean: float = 0.0
    variance: float = 1.0
    # POINT: fixed value
    value: float = 0.0


# ── Desires / objectives ──────────────────────────────────────────

class DesireSpec(BaseModel):
    desire_id: str = Field(default_factory=lambda: str(uuid.uuid4()))
    name: str
    description: str = ""
    # environment key this desire targets
    target_env_key: str
    # direction: +1 = maximize, -1 = minimize
    direction: float = 1.0
    # salience weight (0–1) — how much this desire drives behavior
    weight: float = 1.0


# ── Capabilities ──────────────────────────────────────────────────

class CapabilitySpec(BaseModel):
    capability_id: str = Field(default_factory=lambda: str(uuid.uuid4()))
    name: str
    description: str = ""
    # max capacity units
    capacity: float = 1.0
    # units consumed per use
    cost: float = 0.1
    # recovery rate per tick
    recovery_rate: float = 0.05
    # cooldown ticks after use
    cooldown_ticks: int = 0


# ── Actors ────────────────────────────────────────────────────────

class ActorSpec(BaseModel):
    actor_id: str = Field(default_factory=lambda: str(uuid.uuid4()))
    name: str
    description: str = ""
    # fully-qualified agent class, e.g. "scenarios.hormuz.agents.IranAgent"
    agent_class: str = "core.agents.base.BDIAgent"
    beliefs: list[BeliefSpec] = Field(default_factory=list)
    desires: list[DesireSpec] = Field(default_factory=list)
    capabilities: list[CapabilitySpec] = Field(default_factory=list)
    # initial environment keys this actor owns
    # e.g. {"iran__military_readiness": 0.7, "iran__oil_revenue": 0.4}
    initial_env_contributions: dict[str, float] = Field(default_factory=dict)
    # arbitrary scenario-specific metadata
    metadata: dict[str, Any] = Field(default_factory=dict)


# ── Timeframe ─────────────────────────────────────────────────────

class TimeframeSpec(BaseModel):
    total_ticks: int = 365
    tick_unit: str = "day"   # "hour", "day", "week", "month", "quarter", "year"
    # real-world anchor date (ISO 8601), optional
    start_date: str | None = None


# ── Uncertainty ───────────────────────────────────────────────────
```

```python
class UncertaintySpec(BaseModel):
    # global observation noise applied to all env readings by agents
    observation_noise_sigma: float = 0.02
    # probability of an external shock per tick (0–1)
    shock_probability: float = 0.01
    # max magnitude of a random shock to any single env key
    shock_magnitude: float = 0.1
    # optional named shocks: {tick: {env_key: delta}}
    scheduled_shocks: dict[int, dict[str, float]] = Field(default_factory=dict)


# ── Outcome metrics ──────────────────────────────────────────────

class OutcomeMetricSpec(BaseModel):
    metric_id: str = Field(default_factory=lambda: str(uuid.uuid4()))
    name: str
    description: str = ""
    # environment key to track
    env_key: str
    # optional threshold that triggers a named snapshot when crossed
    snapshot_threshold: float | None = None
    snapshot_direction: float = 1.0  # +1 = above threshold, -1 = below


# ── Research sources ─────────────────────────────────────────────

class ResearchSourceSpec(BaseModel):
    source_id: str = Field(default_factory=lambda: str(uuid.uuid4()))
    source_type: str  # "arxiv", "ssrn", "fred", "world_bank", "news"
    query: str
    # calibration target: which env key this source informs
    calibrates: str | None = None
    # raw data snapshot (filled by research pipeline)
    data_snapshot: dict[str, Any] = Field(default_factory=dict)


# ── Theory references ────────────────────────────────────────────

class TheoryRef(BaseModel):
    theory_id: str   # matches registry key, e.g. "richardson_arms_race"
    priority: int = 0  # lower = runs first in tick loop
    parameters: dict[str, float] = Field(default_factory=dict)


# ── SimSpec (root) ───────────────────────────────────────────────

class SimSpec(BaseModel):
    spec_id: str = Field(default_factory=lambda: str(uuid.uuid4()))
    name: str
    description: str = ""
    domain: str = ""  # "geopolitics", "market", "macro", etc.

    actors: list[ActorSpec] = Field(default_factory=list)
    theories: list[TheoryRef] = Field(default_factory=list)
    timeframe: TimeframeSpec = Field(default_factory=TimeframeSpec)
    uncertainty: UncertaintySpec = Field(default_factory=UncertaintySpec)
    metrics: list[OutcomeMetricSpec] = Field(default_factory=list)
    research_sources: list[ResearchSourceSpec] = Field(default_factory=list)

    # global environment seed values (all floats — no nested objects)
    # key convention: "{theory_id}__{var}", "{actor_id}__{var}", "global__{var}"
    initial_environment: dict[str, float] = Field(default_factory=dict)

    # arbitrary metadata for the scoping agent to attach
    metadata: dict[str, Any] = Field(default_factory=dict)

    @model_validator(mode="after")
    def actor_ids_unique(self) -> "SimSpec":
        ids = [a.actor_id for a in self.actors]
```

```
        if len(ids) != len(set(ids)):
            raise ValueError("actor_ids must be unique")
        return self

    @model_validator(mode="after")
    def metric_env_keys_exist(self) -> "SimSpec":
        env_keys = set(self.initial_environment.keys())
        for metric in self.metrics:
            if metric.env_key not in env_keys:
                raise ValueError(
                    f"OutcomeMetric '{metric.name}' references env_key "
                    f"'{metric.env_key}' which is not in initial_environment"
                )
        return self
```

Key design decisions

Decision	Rationale
`dict[str, float]` environment (no nested objects)	Theories and agents read/write floats by key. Simple, fast, serializable.
UUIDs as strings	JSON-serializable without custom encoders.
`agent_class` as dotted path string	SimRunner imports dynamically — no coupling between spec and implementation.
`initial_env_contributions` on ActorSpec	Actor-owned keys are set at spec time; SimRunner merges them into the global env at setup.
`TheoryRef.priority`	Theories with lower priority run first within a tick. Prevents order-dependence surprises.

---

2. `core/agents/base.py` — BDIAgent

BDI = Belief-Desire-Intention. Beliefs are probabilistic. Decisions derive from expected utility over desires.

```
from __future__ import annotations

import math
import random
from abc import ABC, abstractmethod
from dataclasses import dataclass, field
from typing import Any


# ── Belief types ──────────────────────────────────────────────────────

@dataclass
class BetaBelief:
    """
    Conjugate prior for probability beliefs (0–1).
    alpha = pseudo-count of "successes", beta = pseudo-count of "failures".
    """
    name: str
    alpha: float = 1.0
    beta: float = 1.0

    @property
    def mean(self) -> float:
```

```python
        return self.alpha / (self.alpha + self.beta)

    @property
    def variance(self) -> float:
        a, b = self.alpha, self.beta
        n = a + b
        return (a * b) / (n * n * (n + 1))

    def update(self, observed: float, precision: float = 1.0, bias: float = 0.0) -> None:
        """
        Bayesian update given an observation in [0, 1].
        precision: strength of this observation (effective sample size)
        bias: systematic over/under-estimation (e.g. adversarial perception)
        """
        adjusted = max(0.0, min(1.0, observed + bias))
        self.alpha += precision * adjusted
        self.beta += precision * (1.0 - adjusted)

    def sample(self) -> float:
        """Draw a sample from the Beta distribution."""
        return random.betavariate(self.alpha, self.beta)


@dataclass
class GaussianBelief:
    """
    Kalman-style belief for continuous quantities.
    mean: current estimate, variance: uncertainty.
    """
    name: str
    mean: float = 0.0
    variance: float = 1.0

    def update(self, observed: float, obs_variance: float = 0.1) -> None:
        """
        Kalman filter update step.
        obs_variance: noise in the observation.
        """
        kalman_gain = self.variance / (self.variance + obs_variance)
        self.mean = self.mean + kalman_gain * (observed - self.mean)
        self.variance = (1 - kalman_gain) * self.variance

    def sample(self) -> float:
        return random.gauss(self.mean, math.sqrt(self.variance))


# ── Actions ──────────────────────────────────────────────────────────

@dataclass
class Action:
    action_id: str
    # target actor_id, env key, or "environment"
    target: str
    # signed intensity in [-1, 1]: how strongly to push the target
    intensity: float
    # capability_id required to execute this action
    capability_id: str | None = None
    # env key mutations: {key: delta} — applied by SimRunner after all decide() calls
    parameters: dict[str, float] = field(default_factory=dict)
    description: str = ""


# ── BDIAgent base ────────────────────────────────────────────────────

class BDIAgent(ABC):
    """
    Abstract BDI agent. Subclass and implement decide().

    Lifecycle per tick (called by SimRunner):
        1. observe_environment(env)   — noisy read of env keys
```

```
        2. update_beliefs(observations) — Bayesian update
        3. decide(env, tick)           — abstract: return list[Action]
        4. (SimRunner resolves actions and mutates env)
        5. recharge_capabilities()     — partial recovery
    """

    def __init__(
        self,
        actor_id: str,
        name: str,
        beliefs: dict[str, BetaBelief | GaussianBelief] | None = None,
        desires: list[dict[str, Any]] | None = None,
        capabilities: dict[str, dict[str, float]] | None = None,
        observation_noise_sigma: float = 0.02,
    ) -> None:
        self.actor_id = actor_id
        self.name = name
        self.beliefs: dict[str, BetaBelief | GaussianBelief] = beliefs or {}
        self.desires: list[dict[str, Any]] = desires or []
        # capabilities: {capability_id: {capacity, cost, recovery_rate, cooldown, current, cooldown_remainin
g}}
        self.capabilities: dict[str, dict[str, float]] = capabilities or {}
        self.observation_noise_sigma = observation_noise_sigma
        # last tick's observations (raw, noisy)
        self._observations: dict[str, float] = {}

    # ── Observation ──────────────────────────────────────────────

    def observe_environment(self, env: dict[str, float]) -> dict[str, float]:
        """
        Read environment with Gaussian noise.
        Stores result in self._observations and returns it.
        """
        obs: dict[str, float] = {}
        for key, value in env.items():
            noise = random.gauss(0.0, self.observation_noise_sigma)
            obs[key] = max(0.0, value + noise)
        self._observations = obs
        return obs

    # ── Belief update ────────────────────────────────────────────

    def update_beliefs(self, observations: dict[str, float] | None = None) -> None:
        """
        Update beliefs from observations. Override for custom update logic.
        Default: update any belief whose name matches an observation key.
        """
        obs = observations or self._observations
        for belief_name, belief in self.beliefs.items():
            if belief_name in obs:
                if isinstance(belief, BetaBelief):
                    belief.update(obs[belief_name])
                elif isinstance(belief, GaussianBelief):
                    belief.update(obs[belief_name], obs_variance=self.observation_noise_sigma ** 2)

    # ── Decision ─────────────────────────────────────────────────

    @abstractmethod
    def decide(self, env: dict[str, float], tick: int) -> list[Action]:
        """
        Core BDI decision loop. Return a list of Actions to attempt.
        SimRunner will filter by can_act() before applying.
        """
        ...

    # ── Utility ──────────────────────────────────────────────────

    def expected_utility(self, env: dict[str, float]) -> float:
        """
        Compute expected utility across all desires.
```

```python
        Returns a weighted sum of (direction × current_value) for each desire.
        """
        total = 0.0
        for desire in self.desires:
            key = desire.get("target_env_key", "")
            direction = desire.get("direction", 1.0)
            weight = desire.get("weight", 1.0)
            value = env.get(key, 0.0)
            total += weight * direction * value
        return total

    # ── Capability management ──────────────────────────────────────

    def can_act(self, capability_id: str) -> bool:
        """True if the capability exists, has capacity, and is off cooldown."""
        if capability_id not in self.capabilities:
            return False
        cap = self.capabilities[capability_id]
        return (
            cap.get("current", 0.0) >= cap.get("cost", 0.0)
            and cap.get("cooldown_remaining", 0) <= 0
        )

    def expend_capacity(self, capability_id: str) -> bool:
        """Consume capacity and start cooldown. Returns False if cannot act."""
        if not self.can_act(capability_id):
            return False
        cap = self.capabilities[capability_id]
        cap["current"] = cap.get("current", 0.0) - cap.get("cost", 0.0)
        cap["cooldown_remaining"] = cap.get("cooldown_ticks", 0)
        return True

    def recharge_capabilities(self) -> None:
        """Called by SimRunner at end of each tick."""
        for cap in self.capabilities.values():
            max_capacity = cap.get("capacity", 1.0)
            recovery = cap.get("recovery_rate", 0.05)
            cap["current"] = min(max_capacity, cap.get("current", 0.0) + recovery)
            if cap.get("cooldown_remaining", 0) > 0:
                cap["cooldown_remaining"] -= 1

    # ── Snapshot ──────────────────────────────────────────────────

    def get_state_snapshot(self) -> dict[str, Any]:
        """Return serializable state for snapshot storage."""
        return {
            "actor_id": self.actor_id,
            "name": self.name,
            "beliefs": {
                k: {"mean": b.mean, "variance": b.variance}
                if isinstance(b, GaussianBelief)
                else {"mean": b.mean, "alpha": b.alpha, "beta": b.beta}
                for k, b in self.beliefs.items()
            },
            "capabilities": {
                k: {kk: v for kk, v in cap.items()}
                for k, cap in self.capabilities.items()
            },
        }
```

Key design decisions

Decision	Rationale
`BetaBelief` for probabilities	Natural for "P(actor cooperates)" type beliefs. Conjugate prior = cheap update.

`GaussianBelief` for continuous	Kalman gain handles uncertainty correctly.
`decide()` returns `list[Action]`	SimRunner resolves all actions after all agents decide — no action ordering bias.
Capabilities as plain dicts	Easy to serialize to snapshots without custom `__json__` methods.
`observation_noise_sigma` on agent	Per-agent noise models intelligence differences between actors.

---

3. `core/theories/base.py` — TheoryBase

A theory module is a pure function over the environment: `update(env, agents, tick) → env_delta`. Composable, order-controlled via priority.

```python
from __future__ import annotations

from abc import ABC, abstractmethod
from dataclasses import dataclass, field
from typing import TYPE_CHECKING, Any

from pydantic import BaseModel

if TYPE_CHECKING:
    from core.agents.base import BDIAgent


# ── State variable contract ──────────────────────────────────────────

@dataclass
class TheoryStateVariables:
    """
    Declares which environment keys this theory reads, writes, and initializes.
    Used by SimRunner to validate key conflicts and set up initial environment.
    """
    reads: list[str] = field(default_factory=list)
    writes: list[str] = field(default_factory=list)
    # keys this theory creates at setup (with default values)
    initializes: list[str] = field(default_factory=list)


# ── Theory base ──────────────────────────────────────────────────────

class TheoryBase(ABC):
    """
    Abstract base for all theory modules.

    Subclass, declare state_variables, implement update().

    Convention for env keys owned by this theory:
        "{theory_id}__{variable_name}"
    e.g. "richardson_arms_race__escalation_rate"
    """

    theory_id: str = "base"  # override in subclass

    class Parameters(BaseModel):
        """Override in subclass to define theory parameters."""
        pass

    def __init__(self, parameters: dict[str, float] | None = None) -> None:
        self.params = self.Parameters(**(parameters or {}))
```

```python
        self._tick_history: list[dict[str, float]] = []

    @property
    def state_variables(self) -> TheoryStateVariables:
        """
        Override to declare reads/writes/initializes.
        SimRunner calls this during setup() to validate and seed the environment.
        """
        return TheoryStateVariables()

    def setup(self, env: dict[str, float]) -> dict[str, float]:
        """
        Called once before the tick loop starts.
        Initialize any env keys this theory owns that aren't already set.
        Returns the env keys to set (SimRunner merges them).
        """
        inits: dict[str, float] = {}
        for key in self.state_variables.initializes:
            if key not in env:
                inits[key] = 0.0
        return inits

    @abstractmethod
    def update(
        self,
        env: dict[str, float],
        agents: list["BDIAgent"],
        tick: int,
    ) -> dict[str, float]:
        """
        Core theory update. Called once per tick by SimRunner.

        Args:
            env: current environment (read-only by convention — do not mutate directly)
            agents: all active agents (read beliefs, do not mutate)
            tick: current tick number

        Returns:
            dict of {env_key: new_value} — SimRunner applies these after all theories run.
            Return only keys you own (listed in state_variables.writes).
        """
        ...

    def get_state_snapshot(self) -> dict[str, Any]:
        """Return current theory state for snapshot."""
        return {
            "theory_id": self.theory_id,
            "parameters": self.params.model_dump(),
        }
```

Registry (`core/theories/__init__.py`)

```python
from __future__ import annotations

from typing import Callable, Type

_THEORY_REGISTRY: dict[str, Type["TheoryBase"]] = {}


def register_theory(theory_id: str) -> Callable:
    """Decorator to register a theory class by ID."""
    def decorator(cls):
        cls.theory_id = theory_id
        _THEORY_REGISTRY[theory_id] = cls
        return cls
    return decorator
```

```python
def get_theory(theory_id: str) -> Type["TheoryBase"]:
    if theory_id not in _THEORY_REGISTRY:
        raise KeyError(
            f"Theory '{theory_id}' not registered. "
            f"Available: {list(_THEORY_REGISTRY.keys())}"
        )
    return _THEORY_REGISTRY[theory_id]


def list_theories() -> list[str]:
    return sorted(_THEORY_REGISTRY.keys())
```

Example theory stub

```python
# core/theories/richardson_arms_race.py
from core.theories import register_theory
from core.theories.base import TheoryBase, TheoryStateVariables
from pydantic import BaseModel


@register_theory("richardson_arms_race")
class RichardsonArmsRace(TheoryBase):
    """
    Lewis Fry Richardson (1960) arms race model.
    dX/dt = aY - mX + g   (actor 1)
    dY/dt = bX - nY + h   (actor 2)
    """

    class Parameters(BaseModel):
        a: float = 0.2   # actor1 response to actor2 arms
        b: float = 0.2   # actor2 response to actor1 arms
        m: float = 0.1   # actor1 fatigue/cost coefficient
        n: float = 0.1   # actor2 fatigue/cost coefficient
        g: float = 0.05  # actor1 grievance term
        h: float = 0.05  # actor2 grievance term

    @property
    def state_variables(self) -> TheoryStateVariables:
        return TheoryStateVariables(
            reads=["actor1__military_spending", "actor2__military_spending"],
            writes=[
                "richardson_arms_race__actor1_arms",
                "richardson_arms_race__actor2_arms",
                "richardson_arms_race__escalation_index",
            ],
            initializes=[
                "richardson_arms_race__actor1_arms",
                "richardson_arms_race__actor2_arms",
                "richardson_arms_race__escalation_index",
            ],
        )

    def update(self, env, agents, tick):
        x = env.get("richardson_arms_race__actor1_arms", 0.5)
        y = env.get("richardson_arms_race__actor2_arms", 0.5)
        p = self.params

        dx = p.a * y - p.m * x + p.g
        dy = p.b * x - p.n * y + p.h

        new_x = max(0.0, min(1.0, x + dx * 0.01))
        new_y = max(0.0, min(1.0, y + dy * 0.01))
        escalation = (new_x + new_y) / 2.0
```

```
        return {
            "richardson_arms_race__actor1_arms": new_x,
            "richardson_arms_race__actor2_arms": new_y,
            "richardson_arms_race__escalation_index": escalation,
        }
```

Key design decisions

Decision	Rationale
`update()` returns delta dict, not mutates env	SimRunner can run theories in sequence, accumulate deltas, apply once — no race conditions.
`state_variables.writes` declaration	SimRunner can validate at setup() that no two theories write the same key.
`@register_theory` decorator	Auto-discovery: import the module, it registers itself. No manual registry maintenance.
Parameters as inner Pydantic `BaseModel`	Validated from `TheoryRef.parameters` dict at construction.

---

4. `core/sim_runner.py` — SimRunner

The tick engine. Owns the environment dict and orchestrates all agents and theories.

```
from __future__ import annotations

import asyncio
import random
import threading
import time
from dataclasses import dataclass, field
from typing import Any

from core.agents.base import Action, BDIAgent
from core.spec import ActorSpec, SimSpec, TheoryRef
from core.theories import get_theory
from core.theories.base import TheoryBase


# ── Snapshot types ──────────────────────────────────────────────────

@dataclass
class SimSnapshot:
    tick: int
    label: str
    env: dict[str, float]
    agent_states: list[dict[str, Any]]
    theory_states: list[dict[str, Any]]
    timestamp: float = field(default_factory=time.time)


@dataclass
class MetricRecord:
    tick: int
    metric_id: str
    env_key: str
    value: float
```

```python
# ── Snapshot triggers ──────────────────────────────────────────────────────

@dataclass
class ScheduledSnapshotTrigger:
    """Snapshot every N ticks."""
    interval: int
    label_prefix: str = "auto"

    def should_trigger(self, tick: int, env: dict[str, float]) -> bool:
        return tick > 0 and tick % self.interval == 0

    def label(self, tick: int) -> str:
        return f"{self.label_prefix}_tick_{tick}"


@dataclass
class ThresholdSnapshotTrigger:
    """Snapshot when an env key crosses a threshold."""
    env_key: str
    threshold: float
    direction: float = 1.0  # +1 = above, -1 = below
    label: str = "threshold"
    _triggered: bool = field(default=False, init=False)

    def should_trigger(self, tick: int, env: dict[str, float]) -> bool:
        if self._triggered:
            return False
        value = env.get(self.env_key, 0.0)
        crossed = (self.direction > 0 and value >= self.threshold) or \
                  (self.direction < 0 and value <= self.threshold)
        if crossed:
            self._triggered = True
        return crossed


# ── SimRunner ──────────────────────────────────────────────────────────────

class SimRunner:
    """
    Domain-agnostic simulation engine.

    Usage:
        runner = SimRunner(spec)
        runner.setup()
        runner.run()                    # blocking, runs in thread via asyncio.to_thread
        snapshots = runner.snapshots
        history = runner.metric_history
    """

    def __init__(self, spec: SimSpec) -> None:
        self.spec = spec
        self.env: dict[str, float] = {}
        self.agents: list[BDIAgent] = []
        self.theories: list[TheoryBase] = []
        self.snapshots: list[SimSnapshot] = []
        self.metric_history: list[MetricRecord] = []
        self._lock = threading.Lock()
        self._snapshot_triggers: list[ScheduledSnapshotTrigger | ThresholdSnapshotTrigger] = []
        self._running = False

    # ── Setup ──────────────────────────────────────────────────────────────

    def setup(self) -> None:
        """
        Initialize environment, build agents, instantiate theories.
        Must be called before run().
        """
        # 1. Seed global env from spec
        self.env = dict(self.spec.initial_environment)
```

```python
        # 2. Merge actor-owned env contributions
        for actor_spec in self.spec.actors:
            for key, value in actor_spec.initial_env_contributions.items():
                self.env[key] = value

        # 3. Build agents
        self.agents = [self._build_agent(actor_spec) for actor_spec in self.spec.actors]

        # 4. Instantiate theories in priority order
        sorted_theory_refs = sorted(self.spec.theories, key=lambda t: t.priority)
        self.theories = [self._build_theory(ref) for ref in sorted_theory_refs]

        # 5. Validate no theory write conflicts
        self._validate_theory_conflicts()

        # 6. Run theory setup() — let theories seed their own keys
        for theory in self.theories:
            env_additions = theory.setup(self.env)
            self.env.update(env_additions)

        # 7. Set up snapshot triggers from metrics
        for metric in self.spec.metrics:
            if metric.snapshot_threshold is not None:
                self._snapshot_triggers.append(
                    ThresholdSnapshotTrigger(
                        env_key=metric.env_key,
                        threshold=metric.snapshot_threshold,
                        direction=metric.snapshot_direction,
                        label=f"threshold_{metric.name}",
                    )
                )
        # Default: snapshot every 10% of total ticks
        interval = max(1, self.spec.timeframe.total_ticks // 10)
        self._snapshot_triggers.append(ScheduledSnapshotTrigger(interval=interval))

    def _build_agent(self, actor_spec: ActorSpec) -> BDIAgent:
        """Dynamically import and instantiate the agent class from actor_spec.agent_class."""
        module_path, class_name = actor_spec.agent_class.rsplit(".", 1)
        import importlib
        module = importlib.import_module(module_path)
        AgentClass = getattr(module, class_name)

        # Build beliefs dict
        from core.agents.base import BetaBelief, GaussianBelief, BeliefDistType  # local import avoids circul
ar
        beliefs = {}
        for b_spec in actor_spec.beliefs:
            if b_spec.dist_type == BeliefDistType.BETA:
                beliefs[b_spec.belief_id] = BetaBelief(
                    name=b_spec.name, alpha=b_spec.alpha, beta=b_spec.beta
                )
            elif b_spec.dist_type == BeliefDistType.GAUSSIAN:
                beliefs[b_spec.belief_id] = GaussianBelief(
                    name=b_spec.name, mean=b_spec.mean, variance=b_spec.variance
                )
            # POINT beliefs: stored as GaussianBelief with zero variance
            else:
                beliefs[b_spec.belief_id] = GaussianBelief(
                    name=b_spec.name, mean=b_spec.value, variance=0.0
                )

        # Build capabilities dict
        capabilities = {}
        for cap_spec in actor_spec.capabilities:
            capabilities[cap_spec.capability_id] = {
                "capacity": cap_spec.capacity,
                "cost": cap_spec.cost,
                "recovery_rate": cap_spec.recovery_rate,
                "cooldown_ticks": cap_spec.cooldown_ticks,
                "current": cap_spec.capacity,  # start full
```

```
            "cooldown_remaining": 0,
        }

    return AgentClass(
        actor_id=actor_spec.actor_id,
        name=actor_spec.name,
        beliefs=beliefs,
        desires=[d.model_dump() for d in actor_spec.desires],
        capabilities=capabilities,
        observation_noise_sigma=self.spec.uncertainty.observation_noise_sigma,
    )

def _build_theory(self, ref: TheoryRef) -> TheoryBase:
    TheoryClass = get_theory(ref.theory_id)
    return TheoryClass(parameters=ref.parameters)

def _validate_theory_conflicts(self) -> None:
    """Raise if two theories declare they write the same env key."""
    seen: dict[str, str] = {}  # key → theory_id
    for theory in self.theories:
        for key in theory.state_variables.writes:
            if key in seen:
                raise ValueError(
                    f"Theory write conflict: '{key}' claimed by both "
                    f"'{seen[key]}' and '{theory.theory_id}'"
                )
            seen[key] = theory.theory_id

# ── Tick loop ──────────────────────────────────────────────────────

def run(self) -> None:
    """
    Blocking tick loop. Call via asyncio.to_thread() from async contexts.
    """
    self._running = True
    total_ticks = self.spec.timeframe.total_ticks
    shock_prob = self.spec.uncertainty.shock_probability
    shock_mag = self.spec.uncertainty.shock_magnitude
    scheduled_shocks = self.spec.uncertainty.scheduled_shocks

    for tick in range(total_ticks):
        with self._lock:
            # 1. Apply scheduled shocks
            if tick in scheduled_shocks:
                for key, delta in scheduled_shocks[tick].items():
                    if key in self.env:
                        self.env[key] = max(0.0, min(1.0, self.env[key] + delta))

            # 2. Apply random shocks
            if random.random() < shock_prob and self.env:
                shock_key = random.choice(list(self.env.keys()))
                shock_delta = random.uniform(-shock_mag, shock_mag)
                self.env[shock_key] = max(0.0, min(1.0, self.env[shock_key] + shock_delta))

            # 3. Agents observe (noisy read)
            observations = {
                agent.actor_id: agent.observe_environment(self.env)
                for agent in self.agents
            }

            # 4. Agents update beliefs
            for agent in self.agents:
                agent.update_beliefs(observations[agent.actor_id])

            # 5. Agents decide → collect all actions
            all_actions: list[tuple[BDIAgent, Action]] = []
            for agent in self.agents:
                actions = agent.decide(self.env, tick)
                for action in actions:
                    if action.capability_id is None or agent.can_act(action.capability_id):
```

```python
                all_actions.append((agent, action))

            # 6. Resolve actions → env mutations
            env_delta: dict[str, float] = {}
            for agent, action in all_actions:
                if action.capability_id:
                    agent.expend_capacity(action.capability_id)
                for key, delta in action.parameters.items():
                    if key in self.env:
                        env_delta[key] = env_delta.get(key, 0.0) + delta

            for key, delta in env_delta.items():
                self.env[key] = max(0.0, min(1.0, self.env[key] + delta))

            # 7. Theory updates (in priority order)
            theory_deltas: dict[str, float] = {}
            for theory in self.theories:
                delta = theory.update(self.env, self.agents, tick)
                theory_deltas.update(delta)

            self.env.update(theory_deltas)

            # 8. Record metrics
            for metric in self.spec.metrics:
                if metric.env_key in self.env:
                    self.metric_history.append(MetricRecord(
                        tick=tick,
                        metric_id=metric.metric_id,
                        env_key=metric.env_key,
                        value=self.env[metric.env_key],
                    ))

            # 9. Recharge capabilities
            for agent in self.agents:
                agent.recharge_capabilities()

            # 10. Snapshot triggers
            for trigger in self._snapshot_triggers:
                if trigger.should_trigger(tick, self.env):
                    label = (
                        trigger.label(tick)
                        if isinstance(trigger, ScheduledSnapshotTrigger)
                        else trigger.label
                    )
                    self._take_snapshot(tick, label)

    self._running = False

async def run_async(self) -> None:
    """Run the tick loop in a thread pool to avoid blocking the async event loop."""
    await asyncio.to_thread(self.run)

# ── Snapshot ──────────────────────────────────────────────────────

def _take_snapshot(self, tick: int, label: str) -> SimSnapshot:
    snapshot = SimSnapshot(
        tick=tick,
        label=label,
        env=dict(self.env),
        agent_states=[a.get_state_snapshot() for a in self.agents],
        theory_states=[t.get_state_snapshot() for t in self.theories],
    )
    self.snapshots.append(snapshot)
    return snapshot

def take_named_snapshot(self, label: str) -> SimSnapshot:
    """Manually trigger a named snapshot (e.g. from API endpoint)."""
    with self._lock:
        return self._take_snapshot(
            tick=len(self.metric_history),
```

```
            label=label,
        )

    # ── State ──────────────────────────────────────────────────────

    def get_current_env(self) -> dict[str, float]:
        with self._lock:
            return dict(self.env)

    def is_running(self) -> bool:
        return self._running
```

Key design decisions

Decision	Rationale
`asyncio.to_thread(self.run)`	Tick loop is CPU-bound synchronous. Blocking the async event loop would freeze the FastAPI server.
`threading.Lock` per SimRunner	Multiple API requests can read state (snapshots, current env) while the loop runs.
All action deltas accumulated before applying	Prevents agent ordering from affecting outcomes. All agents see the same pre-tick env.
Theory deltas applied after all agent actions	Theories model background dynamics. Agents react to a stable environment.
Env clamped to `[0.0, 1.0]`	All env values are normalized. Theories and agents work in consistent value range.
`_validate_theory_conflicts()` at setup	Fail fast — a write conflict would cause silent bugs at runtime.

Environment Key Conventions

All environment keys follow a namespacing convention to prevent collisions:

Prefix	Owner	Example
`{theory_id}__`	Theory module	`richardson_arms_race__escalation_index`
`{actor_id}__`	Actor (agent)	`iran__military_readiness`
`global__`	Shared / cross-cutting	`global__trade_disruption_index`

Theories **declare** their keys in `state_variables.writes`. SimRunner validates at setup time.

Module Dependency Graph

```
core/spec.py          (no internal deps — pure Pydantic)
    ↑
core/agents/base.py   (no internal deps — pure dataclass + ABC)
```

```
    ↑
core/theories/base.py (imports BDIAgent for type hint only)
    ↑
core/theories/__init__.py  (registry — imports TheoryBase)
    ↑
core/sim_runner.py    (imports spec, agents.base, theories registry)
```

No circular imports. Each layer only depends on the layer below it.

Next: Theory Stubs (Week 1, Day 4–5)

Five theory stubs to implement (bodies can return `{}` initially, flesh out in Week 2):

Theory ID	File	Domain
`richardson_arms_race`	`core/theories/richardson_arms_race.py`	Conflict escalation
`wittman_zartman`	`core/theories/wittman_zartman.py`	Negotiation / war termination
`fearon_bargaining`	`core/theories/fearon_bargaining.py`	Bargaining model of war
`keynesian_multiplier`	`core/theories/keynesian_multiplier.py`	Macro / fiscal policy
`porter_five_forces`	`core/theories/porter_five_forces.py`	Market competitive dynamics